SPORTS TOURNAMENT RANKING SYSTEM

Project Report — Python Programming

Project topic	Store match results, calculate points and tie-break values, and rank teams
Scoring policy	Win = 3 points, Draw = 1 point, Loss = 0 points
Tie-break policy	Points → Goal difference → Goals scored → Team name
Implementation	Python 3 (console-based program)

Prepared as an academic project report. Sample data and screenshots are illustrative; run the program to capture actual console screenshots.

1. Problem Statement

In a sports tournament, multiple teams play matches and earn points according to match outcomes. Manually tracking wins, draws, losses, goals scored, goals conceded, and team positions can lead to errors. The aim of this project is to build a Python program that records match results, updates each team's statistics, calculates total points and tie-break values, and displays a correctly ordered tournament table according to a defined competition policy.

2. Objectives

• Accept team names and match results as input.

• Validate that each match has two different known teams and non-negative integer scores.

• Calculate matches played, wins, draws, losses, goals for, goals against, goal difference, and points.

• Rank teams using a consistent tie-break policy.

• Display a readable league table and identify the leading team.

3. Assumptions

• Each match has exactly two teams and a final score for each team.

• Team names are unique; names are treated case-insensitively for matching.

• Scores are non-negative whole numbers.

• A win earns 3 points, a draw earns 1 point for each team, and a loss earns 0 points.

• Ranking order: points (descending), goal difference (descending), goals scored (descending), then team name (ascending).

• The sample program processes one tournament run and does not save data to a file or database.

4. Algorithm

1. Read the number of teams and their names.

2. Create a statistics record for every team, initially setting all values to zero.

3. Read the number of matches.

4. For each match, read the two team names and their scores; validate the input.

5. Update matches played, goals for, goals against, wins/draws/losses, and points.

6. Calculate goal difference as goals for minus goals against.

7. Sort teams by points, goal difference, goals scored, and team name using the stated order.

8. Print the final standings and the first-ranked team.

5. Flowchart

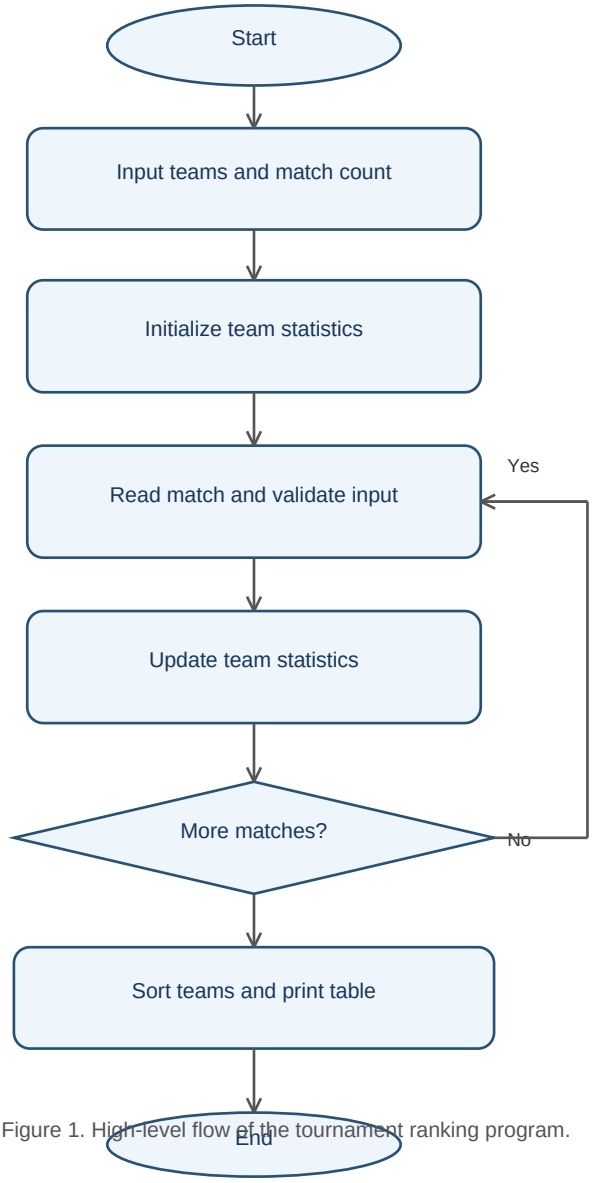

Figure 1. High-level flow of the tournament ranking program.

6. Pseudocode

```
START
  INPUT team_count
  INPUT unique team names
  INITIALIZE each team's statistics to zero
  INPUT match_count

  REPEAT match_count times
    INPUT team_A, score_A, team_B, score_B
    VALIDATE teams are known and different
    VALIDATE scores are non-negative integers
    UPDATE matches played and goals for/against

    IF score_A > score_B
      team_A wins; add 3 points to team_A
      team_B records a loss
    ELSE IF score_A < score_B
      team_B wins; add 3 points to team_B
      team_A records a loss
    ELSE
      both teams record a draw; add 1 point each
    END IF
  END REPEAT

  FOR each team
    goal_difference = goals_for - goals_against
  END FOR

  SORT by points descending,
       goal_difference descending,
       goals_for descending,
       team_name ascending
  DISPLAY standings
  DISPLAY top-ranked team
END
```

7. Python Source Code

```
def read_int(prompt, minimum=0):
    while True:
        try:
            value = int(input(prompt))
            if value < minimum:
                print(f"Enter a number greater than or equal to {minimum}.")
                continue
            return value
        except ValueError:
            print("Invalid input. Enter a whole number.")

def main():
    team_count = read_int("Enter number of teams: ", 2)
    teams = {}
    for i in range(team_count):
        while True:
            name = input(f"Enter name of team {i + 1}: ").strip()
            key = name.casefold()
            if not name:
                print("Team name cannot be empty.")
            elif key in teams:
                print("Team names must be unique.")
            else:
                teams[key] = {
                    "name": name, "played": 0, "won": 0, "drawn": 0,
                    "lost": 0, "gf": 0, "ga": 0, "points": 0
                }
                break

    match_count = read_int("Enter number of matches: ", 0)
    for match_no in range(1, match_count + 1):
        print(f"\nMatch {match_no}")
        while True:
            a = input("Team 1: ").strip().casefold()
            b = input("Team 2: ").strip().casefold()
            if a not in teams or b not in teams:
                print("Both teams must be registered. Try again.")
            elif a == b:
                print("A team cannot play against itself.")
            else:
```

```
                break
        score_a = read_int(f"Goals by {teams[a]['name']}: ", 0)
        score_b = read_int(f"Goals by {teams[b]['name']}: ", 0)

        A, B = teams[a], teams[b]
        A["played"] += 1
        B["played"] += 1
        A["gf"] += score_a
        A["ga"] += score_b
        B["gf"] += score_b
        B["ga"] += score_a

        if score_a > score_b:
            A["won"] += 1
            B["lost"] += 1
            A["points"] += 3
        elif score_a < score_b:
            B["won"] += 1
            A["lost"] += 1
            B["points"] += 3
        else:
            A["drawn"] += 1
            B["drawn"] += 1
            A["points"] += 1
            B["points"] += 1

    ranking = sorted(
        teams.values(),
        key=lambda t: (
            -t["points"],
            -(t["gf"] - t["ga"]),
            -t["gf"],
            t["name"].casefold()
        )
    )

    print("\nTOURNAMENT STANDINGS")
    print("-" * 78)
    print(f"{'Rank':<5}{'Team':<16}{'P':>3}{'W':>3}{'D':>3}"
          f"{'L':>3}{'GF':>4}{'GA':>4}{'GD':>5}{'Pts':>5}")
    print("-" * 78)
    for rank, t in enumerate(ranking, start=1):
        gd = t["gf"] - t["ga"]
        print(f"{rank:<5}{t['name']:<16}{t['played']:>3}"
              f"{t['won']:>3}{t['drawn']:>3}{t['lost']:>3}"
              f"{t['gf']:>4}{t['ga']:>4}{gd:>5}{t['points']:>5}")
    if ranking:
        print(f"\nTop-ranked team: {ranking[0]['name']}")

if __name__ == "__main__":
    main()
```

8. Test Cases

Test	Input / scenario	Expected result
TC-01	A beats B 2–0; B beats C 1–0; A draws C 1–1	A: 4 pts, B: 3 pts, C: 1 pt; ranking A, B, C
TC-02	A and B both have 6 points; A GD +3, B GD +1	A ranks above B by goal difference
TC-03	Two teams have equal points and GD; A scored 5 goals, B scored 3	A ranks above B by goals scored
TC-04	A draws B 0–0	Both teams receive 1 point; GF and GA remain 0
TC-05	Negative score entered	Program rejects it and asks for a non-negative integer
TC-06	Unknown team or same team selected twice	Program rejects the pairing and requests valid teams

9. Sample Output / Console Screenshot

The following is a sample console output for the sample match sequence in TC-01. It is a text mock-up, not a captured screenshot from a running terminal.

```
Enter number of teams: 3
Enter name of team 1: A
```

```
Enter name of team 2: B
Enter name of team 3: C
Enter number of matches: 3

Match 1
Team 1: A
Team 2: B
Goals by A: 2
Goals by B: 0

Match 2
Team 1: B
Team 2: C
Goals by B: 1
Goals by C: 0

Match 3
Team 1: A
Team 2: C
Goals by A: 1
Goals by C: 1

TOURNAMENT STANDINGS
-------------------------------------------------------------------------------
Rank Team              P  W  D  L  GF  GA   GD  Pts
-------------------------------------------------------------------------------
1    A                 2  1  1  0   3   1    2    4
2    B                 2  1  0  1   1   2   -1    3
3    C                 2  0  1  1   1   2   -1    1

Top-ranked team: A
```

For actual submission screenshots: run the Python file in IDLE, VS Code, or a terminal; enter the sample values; capture the code window and the output window using your computer's screenshot shortcut. Replace this mock-up with your own screenshots if your instructor requires real screenshots.

10. Results and Discussion

The program computes standings from match outcomes rather than relying on manually entered totals. Its tie-break sequence gives a deterministic order: points are considered first, followed by goal difference, goals scored, and alphabetical team name. Input checks prevent common data-entry mistakes such as negative scores, unknown teams, duplicate team names, and a team playing itself.

11. Conclusion

The Sports Tournament Ranking System demonstrates the use of dictionaries, loops, conditional statements, input validation, sorting, and formatted output in Python. It automates the calculation of team statistics and produces a clear ranking table under a stated competition policy. Future improvements could include saving results to CSV, loading fixtures from a file, supporting head-to-head tie-break rules, and creating a graphical user interface.

12. How to Run

- Copy the Python source code into a file named sports_ranking.py.
- Run it with Python 3: python sports_ranking.py
- Enter at least two team names, then enter match details when prompted.
- Compare the displayed table with the expected results in the test cases.